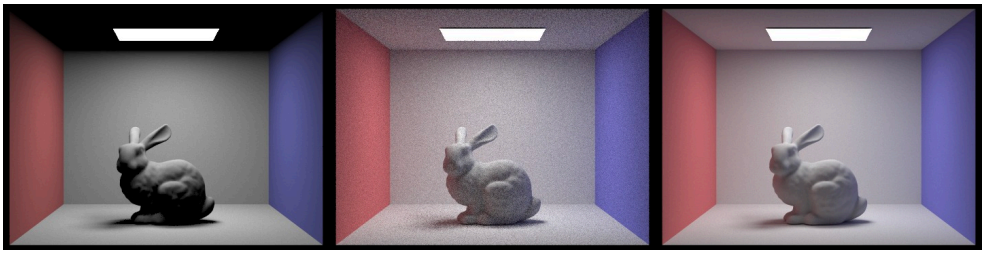
计算机图形学作业3 – 渲染器



在本次图形学作业中，你将探索课程中的渲染主题。你将使用路径追踪算法（Path Tracing）实现一个基于物理的渲染器（Physically-based renderer）。这项作业将帮助你加深理解课堂上讲解的许多概念，包括光线和场景的相交、空间加速数据结构以及基于物理的照明和材质模型等。完成作业后，你将能够生成一些具有照片级真实感的图片，如上图。

本次作业比之前作业需要的时间更多，而且渲染上图所示图像至少需要几个小时，所以请尽早开始！请大家自行完成编程，作业中遇到问题，可与其他同学进行技术交流，但请勿直接抄袭 代码。

源代码的开发和编译方法见另一文件[`计算机图形学作业编译说明.pdf`]。代码中使用CGL库进行向量和矩阵运算，使用方法见[`CGL Vectors Library.pdf`]。

在实现代码的同时，请同时完成作业报告。我们给出了报告模板 `作业报告.docx` ，请按照附加提示完成写作。

作业结构

此次作业分为4个部分。

- 第 1 部分：光线生成与场景求交
- 第 2 部分：空间加速数据结构（以BVH为例）
- 第 3 部分：直接光照
- 第 4 部分：全局光照

如何运行可执行文件

本作业代码编译后生成可执行文件 `pathtracer` ，运行此程序时一般需要指定某个网格的COLLADA文件(.dae)。当未输入COLLADA文件时，将会弹出以下GUI，你将指定相机、窗口和路径追踪的参数，将保存成默认值。



COLLADA 文件基于 XML 格式来定义场景图（与 SVG 文件非常相似），描述了场景中所有对象（网格、相机和灯光等），以及这些对象的坐标变换信息。

你可以使用如下命令运行文件，命令行参数（选项）表示 `pathtracer` 程序将打开 CBspheres_lambertian.dae 文件，并采用 8 个线程来进行渲染。

```
./pathtracer -t 8 ../dae/sky/CBspheres_lambertian.dae
```

与之前的作业不同，我们提供了无窗口运行模式。可通过命令行选项 `-f` 标记来指定渲染图像的名称，这样程序将不打开GUI，运行结束后将渲染的图像保存到指定的文件名。

更具体的运行命令如下，意思是将渲染图像直接保存到*spheres_64_16_6.png*文件，图像分辨率为480x360，每个像素使用64个采样点进行渲染、光线路径深度最多6次反射及每个光源采样16个样本点。s

```
./pathtracer -t 8 -s 64 -l 16 -m 6 -r 480 360 -f spheres_16_4_6.png ../dae/sky/CBspheres_lambertian.dae
```

快速测试渲染结果的方法

高质量渲染需要很长时间才能完成！在程序开发和测试期间，不要花很长时间等待全尺寸、高质量的图像。

以下是获取快速测试图像的一些方法：

- 使用较少的样本进行渲染！

- 使用 -r 标志设置较小的窗口大小（例如 `./pathtracer -t 8 -s 64 -r 120 90 ../dae/sky/CBspheres_lambertian.dae`）。调整分辨率确定整个场景在窗口中可见既可，然后按 `R` 键开始渲染。
- 利用单元格渲染功能——使用 `R` 键开始渲染，然后点击 `C` 键，并用鼠标在GUI上拖动一个边框。此时渲染器将仅渲染你选择的区域内的像素。

使用可执行文件（命令行模式）和图形用户界面(GUI)

命令行选项

标志和参数	描述
`-s <INT>`	每个像素的采样光线数量（默认 = 1，一般设为2的幂）
`-l <INT>`	每个面积光源（area light）的采样数量（默认 = 1）
`-t <INT>`	渲染线程数（默认值 = 1）
`-m <INT>`	最大光路深度（反射次数）（默认值 = 1）
`-o <INT>`	是否累积光路（默认值 = 1， 非零的话则 `isAccumBounces=true`，若是 0 的话， 则仅返回指定的光路深度（既光第N次反弹） 对应的光线辐射率）
`-f <FILENAME>`	无窗口模式下保存输出的图像（.png）文件
`-r <INT> <INT>`	输出图片分辨率（无窗口模式）或GUI窗口的宽度和高度
`-p <x> <y> <dx> <dy>`	与 -f 标志（无窗口模式）一起使用， 指定加速渲染的单元格边框大小，边框的左上角位于 [x,y] 且长宽为 [dx, dy] 像素。
`-c <FILENAME>`	加载相机设置文件（主要是在无窗口模式下设置相机位置）
`-a <INT> <FLOAT>`	指定每个batch的采样样本数量和采样公差(与方差相关的量)
`-H`	启用半球采样进行直接照明
`-h`	打印命令行帮助信息

移动相机（在编辑和BVH模式下）

鼠标	响应
点击左键并拖动	旋转场景
点击右键并拖动	平移场景
滚动键	放大和缩小场景

鼠标	响应
空格键	重置相机

键盘命令

按键	响应
E	网格编辑模式（默认）
V	BVH 可视化模式
←/→	下降到左/右子节点（BVH可视化模式下）
↑	向上移动到父节点（BVH可视化模式下）
R	开始渲染
S	保存窗口截图
-/+	减少/增加面积光源样本
[/]	减少/增加每个像素的采样光路数量
</>	减少/增加最大光路深度
C	切换到单元格窗口渲染模式
H	切换到半球均匀采样方法
D	将相机设置保存到文件

在加速渲染小节我们提到，进入单元格渲染模式，你将使用鼠标突出显示感兴趣的区域，在你更改采样光路数量、光路深度等参数时，更快速的看到渲染结果。

代码结构

先了解下代码的运行逻辑。当调用*pathtracer时，程序的启动流程如下：*

1. *首先main.cpp中的* `main()` 函数会调用*collada/collada.h*文件中的 `ColladaParser` 函数来解析场景文件，建立网格数据结构和相机及渲染参数。
2. 创建 `Viewer` 和 `Application`，也就是创建主程序类和可视化类。`Viewer` 管理底层 OpenGL 细节和创建窗口，其接收鼠标键盘事件并传递给 `Application`。`Application` 创建 `pathtracer` 进程，包含了相机和场景信息。
3. `viewer.start()` 启动进程，等待GUI各种输入。其中最重要的是启动 `set_up_pathtracer()`，调用 `PathTracer::start_raytracing()`。
4. `set_up_pathtracer()` 设置了相机和场景，此部分可能对场景构建空间加速结构，既调用 `PathTracer::build_accel()` 来设置 BVH。
5. 在 `start_raytracing()`（在*pathtracer.cpp*中实现）中，为了并行计算，一般将场景划分为“图块（tiles）”，组成一个图块队列，由 `numWorkerThreads` 进行并行处理。
6. 在队列为空之前，每个线程都会从队列中取出图块并运行 `raytrace_tile()` 进行渲染。`raytrace_tile()` 调用 `raytrace_pixel()` 对图块范围内的每个像素进行

渲染。结果保存到pathtracer的`sampleBuffer`中（`HDRImageBuffer`的实例，定义在*image.h*）。

你将实现物理渲染器的大部分的核心算法，例如

1. 在`raytrace_pixel()`里面，你将编写一个循环，调用`camera->generate_ray(...)`来获取光线，并通过`est_radiance_global_illumination(...)`获取沿着这些光线的辐射度。
2. 在`est_radiance_global_illumination`中，你将使用`bvh->intersect(...)`判断光线和场景是否有交点。如果相交，你将返回值光路累积的辐射率`Spectrum L_out`，光线累积包括两个部分，
   - 在光源直接照射，在，`zero_bounce_radiance`里实现`bsdf->get_emission()`来获取光的辐射率
   - 在全局光照下，实现`at_least_one_bounce_radiance`，它调用了`one_bounce_radiance`（直接光照函数），并递归调用自身。

在以此核心渲染函数之外，你将实现光线与三角形、球体和包围盒相交的函数、构建和遍历 BVH 的函数以及从各种 BSDF 中进行采样的函数。

综上，你将（至少）编辑以下文件

- *pathtracer/pathtracer.cpp*（第1、3、4部分）
- *pathtracer/camera.cpp* (第 1 部分)
- *scene/triangle.cpp* (第 1 部分)
- *scene/sphere.cpp* (第 1 部分)
- *scene/bvh.cpp* (第 2 部分)
- *scene/bbox.cpp*（第 2 部分）
- *pathtracer/bsdf.cpp*（第 3 部分）

下列文件与代码实现的相关性较高，你将使用其中定义的类和函数。

- *pathtracer/ray.h*
- *pathtracer/intersection.h*
- *pathtracer/sampler.h*
- *pathtracer/sampler.cpp*
- *util/random_util.h*
- *scene/light.h*
- *scene/light.cpp*

另外，*ray.h*包含一个已定义的变量`PART`。当前设置为 1。如果你喜欢独立测试各个部分，可以使用此变量。例如，你可以编写类似`if (PART != 3) { return false; }`来控制仅执行至第 3 部分内容。

场景编辑器和GUI功能测试器

该项目附带一个可选的场景编辑器和功能测试器，名称为 Visual Debugger。我们载入的场景文件包含了光源、对象等信息，其按照某种方式排列好，可以使用场景编辑器对这些对象进行编辑。首先你可以查看场景层次结构、移动场景中对象、修改BSDF、编辑光源等。在功能测试器中，我们设置进行了两个测试：一个用于`sample_L`对光源进行采样测试；另一个用于光线与三角形或球相交测试。这些测试对于验证你是否正确实现光线场景相交功能非常有用。

你可以将`src/application/visual_debugger.cpp`文件的第 25 行取消注释以启用此功能：

```
// #define ENABLE_VISUAL_DEBUGGER
```

运行程序时，它将弹出一个单独的窗口。你可以查看这些基于 ImGUI 库的 GUI 测试函数是如何编写的，见 `src/application/visual_debugger.cpp` 并在其中添加你自己的测试函数。这种可视化工具对于debug非常有帮助。

如果在场景中移动灯光，则需要同时移动灯光本身及光源的网格（定义了自发光的BSDF），因为灯光和灯光网格是分开的。如果你只移动其中一个，将出现不正确的照明。

第 1 部分：光线生成与场景求交

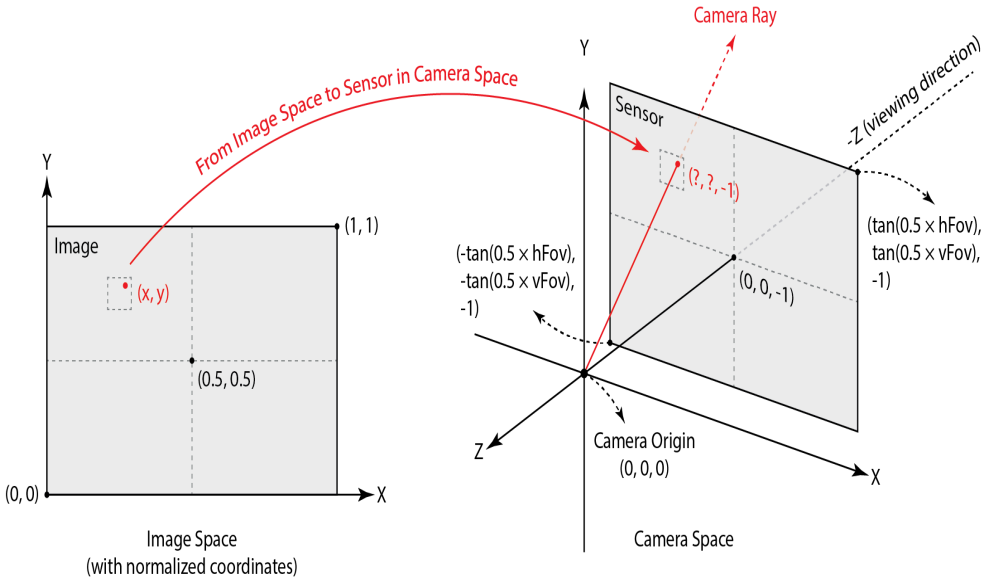
1.1 生成相机射线（Camera Rays）

请在*src/pathtracer/camera.cpp*中完成 `Camera::generate_ray(...)` 函数。

此函数输入**规范化（标准化）**的图像空间坐标 (x, y)，输出在**世界坐标系（world space）**下的光线 `Ray`。这部分需要注意不同坐标系变换，参见lecture 4 Transform，理解世界坐标到屏幕坐标系之间的一系列变换。此处实现分三步，首先将图像空间的坐标转换到**相机空间，然后在相机空间**中生成射线（相机中心和图像坐标的连线），最后将射线转换到**世界空间**中。

以相机为中心的相机坐标系如下图所示，相机位于原点 $(0, 0, 0)$处，相机正面朝向 $-Z$ 轴，该朝向也称为观看方向（Viewing Direction）。

为了模拟相机成像过程，在相机观看方向上，我们定义“**虚拟相机传感器**”（根据小孔成像原理，相机的真实传感器在相机观看方向的反方向上，且呈倒立姿态，为了方便理解和计算，在位于真实传感器的对称位置上定义虚拟传感器，二者成像内容一致，互为倒立），其为长方形平面，垂直于 Z 轴，具体为$Z = -1$。传感器的中心位于$(0, 0, -1)$。其左下角顶点的坐标为$(-tan(\frac{hFov}{2}), -tan(\frac{vFov}{2}), -1)$，右上角顶点坐标为$(tan(\frac{hFov}{2}), tan(\frac{vFov}{2}), -1)$，其中 $hFov$ 和 $vFov$分别为 X 和 Y 轴向的视场角（field of view angles）。FOV的定义见lecture 4 中关于视锥（View frustum）的描述。



这个虚拟相机传感器对应一张图像，反之亦然。具体而言，**规范化**的图像坐标 $(0, 0)$ 和 $(1, 1)$ 应分别映射到相机传感器的左下角和右上角。 注意规范化的图像坐标为0到1的非负数。

如图所示，射线起点为**相机空间**中的相机坐标（原点），并穿过传感器上的顶点（对应于规范化图像坐标(x, y)） 。接下来需要将射线从相机坐标系转换到世界坐标系。射线包括原点和方向，你需要同时对这对顶点和向量进行变换。需要注意的是射线方向也需要进行归一化（模长为1）。

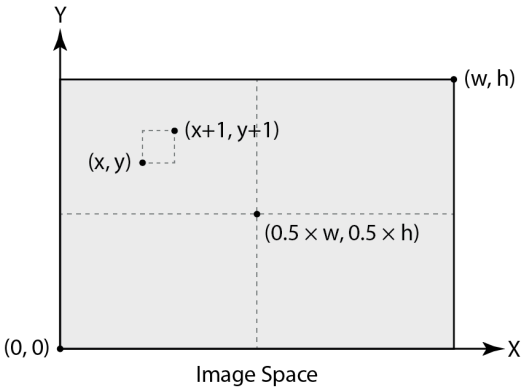
实现细节说明

`Camera`类的下列主要成员变量你在实现中将用到。当然以下列表并不完整，你还可能使用到其他成员变量。

- `hFov`和`vFov`定义了虚拟相机传感器，注意它们的单位是角度，而**不是**弧度。
- `pos`是相机在世界坐标系中的位置， `c2w`是相机到世界的旋转矩阵。
- `nclip`和`fclip`的取值大于零，分别代表近裁剪平面和远裁剪平面。在裁剪平面之外的物体对相机不可见。FOV和远近平面事实上界定了视锥范围。你将根据`nclip`和`fclip`来初始化光线`Ray`的最近和最远点，既`min_t`和code>max_t。1.3节中解释此初始化计算过程。

1.2 生成像素样本

请在*src/pathtracer/pathtracer.cpp*中完成`PathTracer::raytrace_pixel(...)`函数。



与上一节规范化的图像空间坐标不同的是，图像的像素坐标位于**非归一化的图像空间**中，为离散数值，和图像分辨率相关。函数输入为像素坐标(x, y)，并在`sampleBuffer`中更新该像素的辐射度积分结果，对于颜色图，像素有RGB三个通道，所以积分结果为`Vector3D`对象`ns_aa`。

要估计像素的辐射度积分，应该随机生成`ns_aa`条射线，方法见1.1节的`generate_ray(...)`。对于每条射线，应调用`PathTracer::est_radiance_global_illumination(Ray r)`函数来估计该光路的辐照度，再通过Monte Carlo估计把所有光线进行加权求和，得到`Vector3D`类型的像素。

采样器简介

采样是图形学课程中的重要概念，lecture 13 Monte Carlo Ray Tracing就是围绕随机采样来估计辐照度积分结果，本次作业将大量使用这个概念。为此，我们在*src/pathtracer/sampler.h*中提供了丰富的“采样器”类，你可以使用它们从给定的分布中随机采样二维或三维样本。例如， `PathTracer`有一个成员变量`gridSampler`，它表示在正方形区域内进行均匀采样。对于任何采样器类，你可以调用`get_sample()`来得到一个随机样本。

实现细节说明

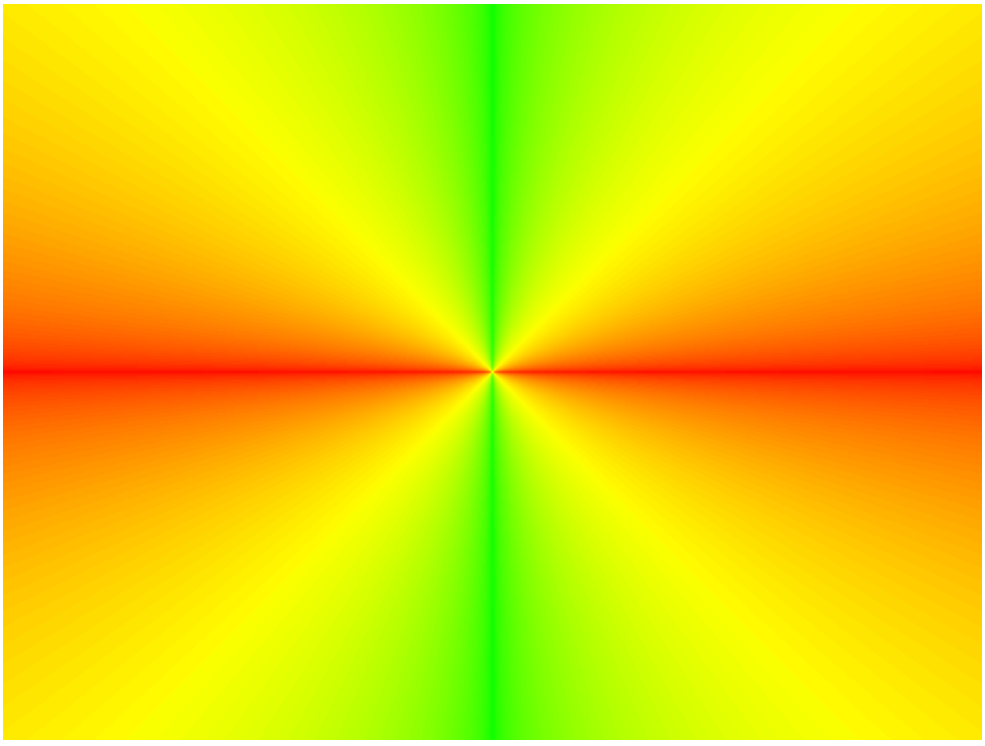
- `PathTracer` 有一个成员变量 `camera` 。你可以调用 `camera->generate_ray(double x, double y)` 来生成一条射线。请注意，此函数的输入坐标(x, y)必须是**规范化坐标**，即 $(x, y) \in [0, 1] \times [0, 1]$。]你可以使用 `sampleBuffer` 的成员变量 `w` 和 `h` 对坐标进行相应的缩放。
- 在初步实现时，你可以让 `PathTracer::est_radiance_global_illumination(...)` 只返回顶点的某类属性，用于进行代码调度，例如可以返回 (1) 相机光线的方向或 (2) 相机光线与物体表面交点的法向。你将在1.3节和1.4节中实现三角形或球体与光线的求交运算，到时候可以获得表面交点的法向。随着作业的深入，你将不断完善该全局照明函数，实现更符合物理过程的真实感照明。
- 你可以使用 `sampleBuffer` 中的 `update_pixel(...)` 方法来更新 `Vector3D` 类型的像素值。

代码测试

完成1.2节后，应该能够运行以下命令来测试你的实现效果。注意，每次命令行改变时，如果使用IDE，你将需要根据你的IDE调整命令行选项，包括*.dae文件*的路径等。

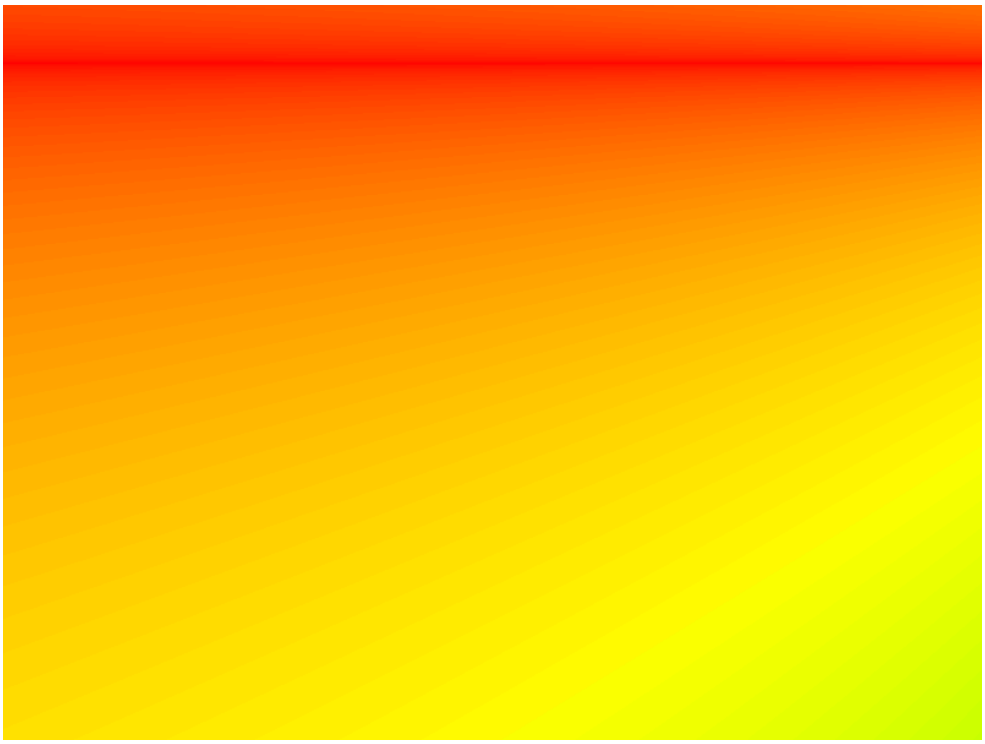
```
./pathtracer -r 800 600 -f CBempty.png ../dae/sky/CBempty.dae
```

上述命令将渲染的图像保存为*CBempty.png*，效果如下图所求。每个像素的RGB值直接可视化了穿过该像素的相机光线方向（X映射到红色，Y映射到绿色，Z映射到蓝色）。**将数据直接可视化为颜色是非常直观的，这是一种常见的调试图形程序的好方法。**



使用不同的场景，其FOV可能不同，生成的光线也将不同。如运行下面命令，生成的图像应该如下图所示。

```
./pathtracer -r 800 600 -f banana.png ../dae/keenan/banana.dae
```



1.3 射线与三角形求交

请在*src/scene/triangle.cpp*中实现 `Triangle::has_intersection(...)` 和 `Triangle::intersect(...)` 两个函数。

`Triangle::has_intersection(...)` 只是判断三角形和射线之间是否有交点。`Triangle::intersect(...)` 不仅测试交点的存在性，若存在还返回最近的交点的位置及其他信息。你可以使用lecture 12 Ray Tracing中介绍的任何求交方法来实现。

在课程中，我们以参数化方程来描述射线，参数为t并且仅$t \geqslant 0$时才是有效的。视锥的远近可视平面进一步限定了参数的取值范围，其中 `min_t` 和 `max_t` 为 `Ray` 的最小和最大值，落于期间的交点才是有效的。当计算射线与三角形（和球体）相交时，你将始终更新 `max_t` 为最小的值（既最近的交点），而其他比该值大的点将被忽略。

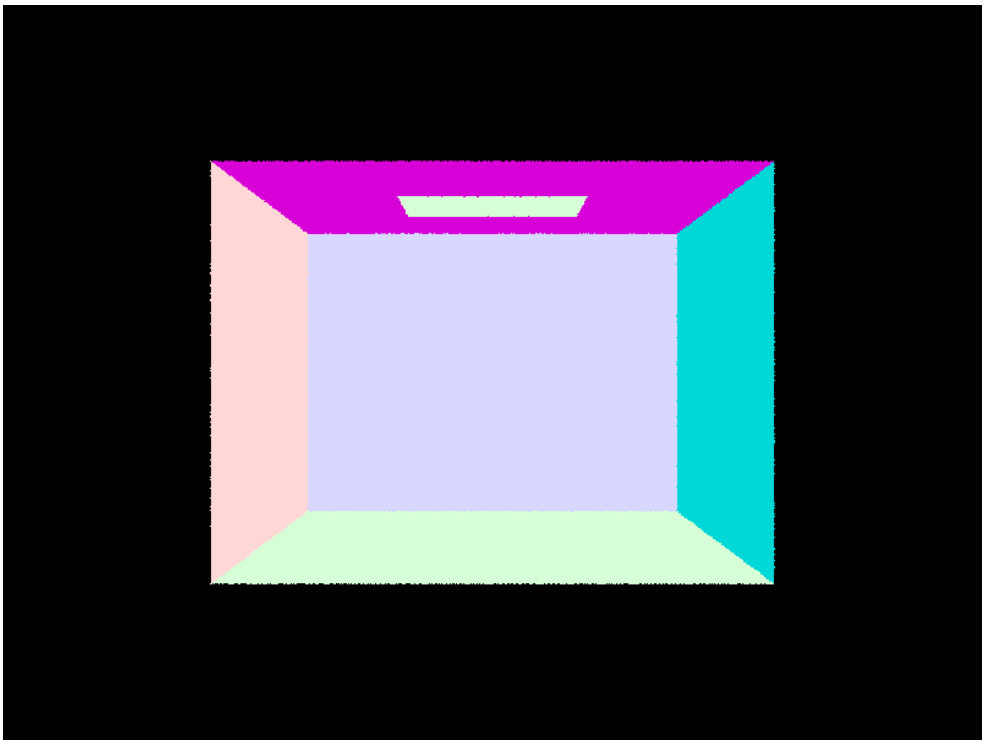
对于 `Triangle::intersect(...)`，如果存在有效交点，则将其记录在交点的数据结构 `Intersection *isect` 中，该结构体内容如下：

- `t` 是输入射线与场景的交点对应的参数t的取值。
- `n` 是交点处的表面法线。因为交点一般不在三角形顶点上，计算三角形内任何一个点的法线属性，应使用重心坐标对三角形的三个顶点法线的进行线性插值。重心坐标的计算方法见lecture 5 shading。
- `primitive` 为指针，其指向与射线相交的图元（使用 `this` 指针）。
- `bsdf` 为指针，指向交点处的表面材质（BSDF）（使用 `get_bsdf()` ）。BSDF 指 Bidirectional Scattering Distribution Function，双向散射分布函数，是 BRDF 的拓展，同时考虑反射和透射现象。

代码测试

完成1.3节后，使用以下命令渲染来渲染图像，效果应如下图所示。

```
./pathtracer -r 800 600 -f CBempty.png ../dae/sky/CBempty.dae
```



1.4 射线与球体求交

请在*在src/scene/sphere.cpp*中完成 `Sphere::has_intersection(...)` 和 `Sphere::intersect(...)` 两个函数。

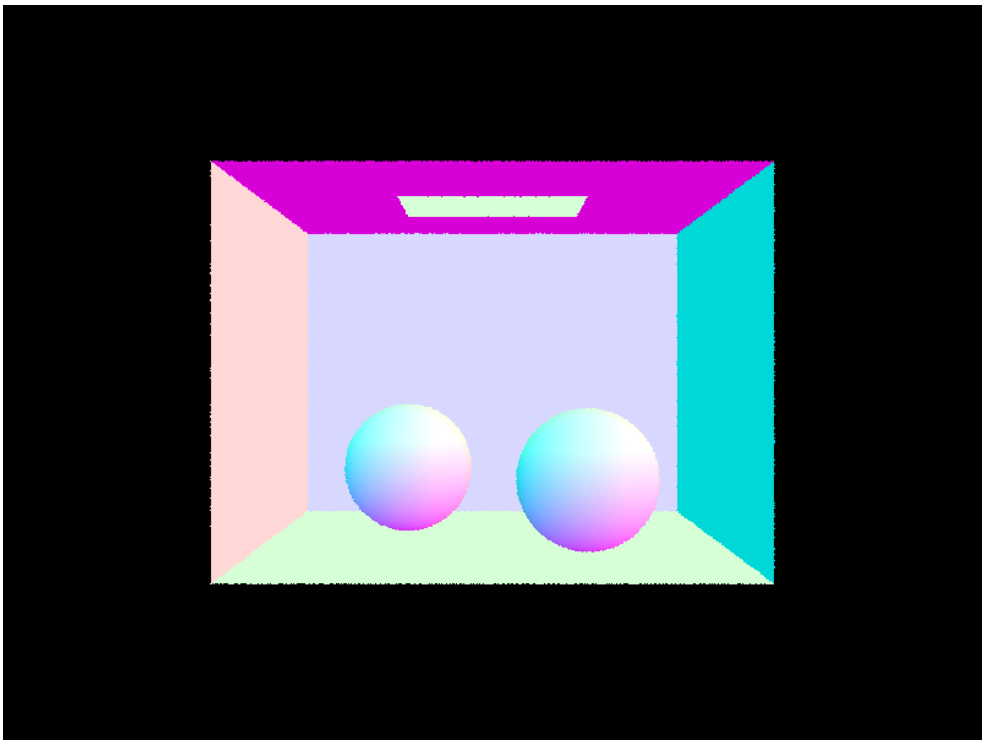
与1.3节一样，对于这两个函数，仅在交点存在时返回 `true` 。交点的计算参见lecture 12 ray tracing课件。 同样，参数*t*应落在 `min_t` 和 `max_t` 之内，当有有效交点时，更新 `max_t` 。值得一提的是，射线与球体相交可能会产生两个交点，你需要选择一个最接近的且有效的交点。

对于 `Sphere::intersect()` ，当有效交点存在，则应填写交点的数据结构 `Intersection *isect` 。与三角形不同的是，球体上的表面法线可以通过解析法求得：法线为从球体中心指向表面交点的射线，并进行归一化。

代码测试

完成1.4节后，使用以下命令渲染图像，效果如下图所示。

```
./pathtracer -r 800 600 -f CBspheres.png ../dae/sky/CBspheres_lambertian.dae
```



第 2 部分：空间加速数据结构（以BVH为例）

时间复杂度试验

你可能已经注意到，即使使用最基本的法线着色，当你输入的.dae文件中的几何图形仅具有稍微复杂的结构时，都需要很长时间才能完成渲染。在第2部分中，我们将实现 Bounding Volume Hierarchy（BVH）空间加速数据结构来加速我们的路径追踪器，使它可以更高效地渲染这些*.dae 文件*。

你可以尝试运行以下网格：

```
./pathtracer -t 8 -r 800 600 -f cow.png ../dae/meshedit/cow.dae
```

这将需要相当长的时间才能完成！使用第 1 部分的求交方案，你可能需要几十秒时间。渲染需要这么长时间的原因是上一节代码只创建一个单个节点的 BVH，将所有三角形和球体直接存储在叶节点中。 我们将看到，仅使用简单的启发式方法构建的 BVH 就可以表现得更好，相交计算的复杂度从$O(n)$ 减小到$O(log(n))$ 。

2.1 构建BVH

请在*src/scene/bvh.cpp*文件中完成 `BVHAccel:construct_bvh(...)` 函数。

BVH 是二叉树。类 `BVHAccel` 本身只包含根节点 `BVHNode *root` 。树中的每个节点包含 (1) 包围盒 `bb` 、(2) 左子节点 `l` 和右子节点 `r` ，以及 (3) 两个STL迭代器 `start` 和 `end` ，分别指向场景图元列表的起点和终点。对于中间节点， `l` 和 `r` 为非 `NULL` （非空）；对于叶子节点， `start` 和 `end` 为非 `NULL` 。我们通过递归的方式来构建BVH。

首先，计算图元集合的包围盒，并初始化一个新的 `BVHNode` 节点，更新其包围盒。当新节点包含的图元数量小于 `max_leaf_size` ，则其为叶节点，其左右子节点当为空，而 `start` 和 `end` 迭代器则应适当更新，指向图元列表的正确区间。最后返回此叶节点并结束递归。

当新节点下的图元数量大于阈值，此时需要将图元划分为“左”和“右”集合。注意，图元的包围盒存在于3D空间中，我们希望找到最优的轴进行分割，并沿着这个轴计算具体的分割点，将图元划分为“左”和“右”集合。具体的分割策略可参考lecture 12中介绍的几种原则和代价计算。对于新创建的左右子节点，分别通过递归调用`BVHAccel:construct_bvh(...)`函数。当所有子节点为叶节点时，返回BVH树。。

你将使用到的函数如下：

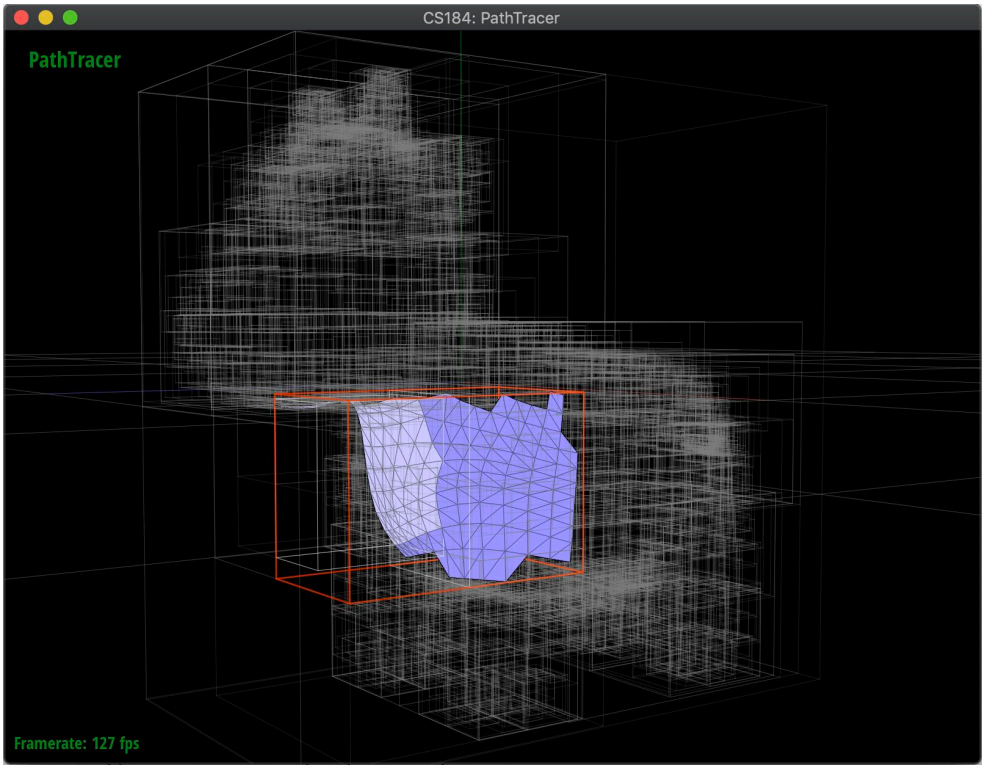
1. `Primitive::get_bbox()`返回图元的包围盒。
2. `Primitive::get_bbox().centroid()`返回图元包围盒的质心。
3. `BBox::expand(...)`扩大包围盒，其根据输入的顶点或其他包围盒，扩大原有的包围盒以容纳之。输入类型为`Vector3D`或`BBox`。

实现细节说明

- 分割点，建议采用轴的中点或图元质心的平均值。
- 如果选择的分割点使得所有图元仅位于分割点的一边，则递归将无限调用，从而出现段错误。
- 你可以尝试其他 BVH 构造方法。一些想法包括更改启发式策略（例如，使用图元集合的中值点等）或改进 BVH 的算法或内存效率。

代码测试

我们的GUI 提供了一个好用的 BVH 可视化模式，你可以使用它来调试你实现的代码。你可以按`V`键进入此模式，可视化从 BVH 的根节点开始展示。在任何节点处，你都可以使用`←`、`→`或`↑`导航到左子节点、右子节点或返回父节点。BVH可视化模式如下图所示：



2.2 射线与包围盒求交

请在*src/scene/bbox.cpp*中完成`BBox::intersect(...)`函数。

此函数的输入为射线，射线经过包围盒内的线段的两个端点分别为`t0`和`t1`（也就是课件中的tmin和tmax）。射线和轴对齐平面的相交方程，以及射线和包围盒相交方法见

lecture 12。

2.3 射线与BVH相交

请在*src/scene/bvh.cpp*中实现`BVHAccel::has_intersection(...)`和`BVHAccel::intersect(...)`函数。

`BVHAccel::has_intersection(...)`仅判断输入射线与输入 BVH 中的**任何**图元之间是否有交点，而`BVHAccel::intersect(...)`不仅测试相交，还将交点存储在结构体`Intersection *i`中。

提供的原始代码假设根节点是叶节点，直接对节点中的每个图元测试相交。实现BVH后，需要对BVH进行递归遍历。

实现细节说明

- 对于`BVHAccel::has_intersection(...)`，当仅有一个交点时，可安全地返回`true`。但对于`BVHAccel::intersect(...)`,，你必须检查`BBox`里与射线相交的每个图元，并返回最近的交点。
- 当射线与`BBox`的交点不在参数`t`的有效区间`min_t`和`max_t`内，则可安全返回`false`。

代码测试

运行下述命令行，重新渲染之前相同的网格：

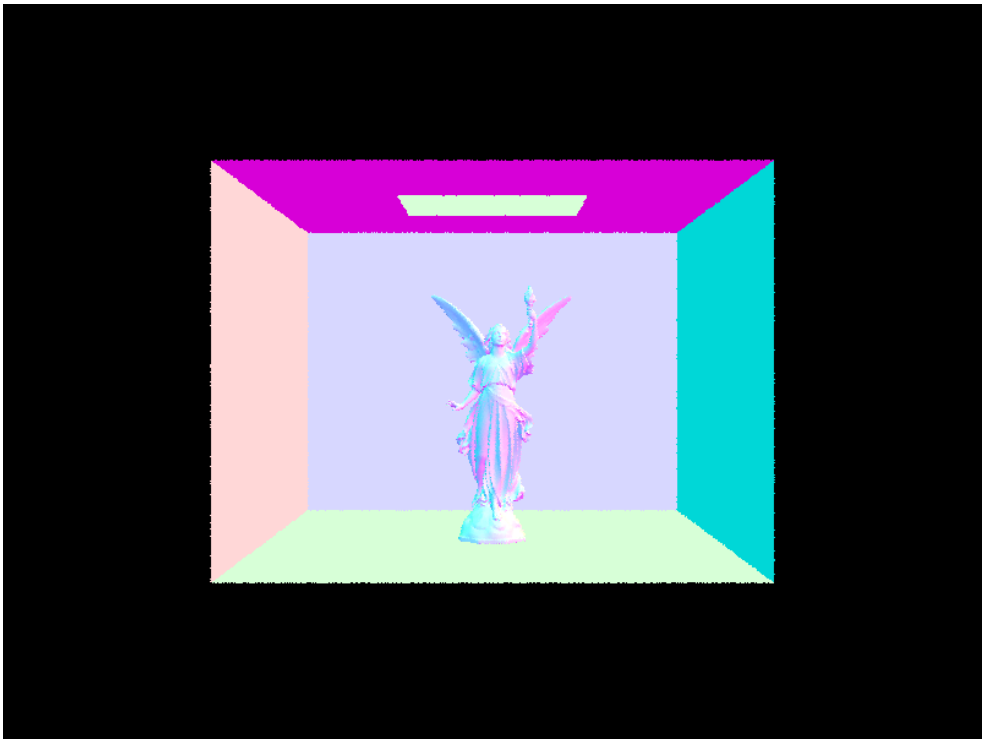
```
./pathtracer -t 8 -r 800 600 -f cow.png ../dae/meshedit/cow.dae
```

新的方案将在毫秒级完成渲染，而之前可能需要几十秒。

2.3节完成后，你的路径追踪器应该足够快，可以在秒级内跑完任何的场景文件（仅使用正常着色模式）。下图所示人像*dae/meshedit/maxplanck.dae*包含了数万个三角形：



或者下图的雕塑*dae/sky/CBlucy.dae*文件则包含了几十万个三角形，依然可以在秒级内完成。



第 3 部分：直接光照

从本部分中，我们将开始模拟光在场景中的传输过程，并使用基于物理的渲染方程来计算图像。

整个渲染流程调用的主要函数是`est_radiance_global_illumination`，位于*src/pathtracer/pathtracer.cpp*文件中。在给定的原始代码中，此函数返回一个颜色用于调度实验，其中RGB设置为表面法线。接下来的几个算法将逐步修改此函数。

3.1 漫反射BSDF

BSDF是BRDF的拓展，描述入射光线与物体表面相互作用后离开物体表面的光量比重，该比重与入射光和出身光的角度有关，也与物体材质相关。材质在课程lecture 11 the rendering equation中重点介绍了。

BSDF 简介

表面的材质属性定义在类`BSDF`中，它描述了光进入物体表面后的反射和透射光线分布情况。具体而言，`BSDF`描述的是**入射光线**从**入射方向向出射方向散射出去的*比重***。 不同类型的材质对应于不同的BSDF子类。

在此任务中，你将实现表示漫反射材的BSDF函数`DiffuseBSDF::f`，该材质在半球上向所有方向均匀反射光，是最简单的一种材质。在*src/pathtracer/bsdf.h*中，我们定义了一些数学辅助函数，用于处理半球内的向量（光线方向）运算。

`DiffuseBSDF::f`的输入为立体角`wi`和立体角`wo`，返回`f(wi -> wo)`。

注意：

- 材质的反光率（albedo）存储在该类的`reflectance`参数中。该参数类型为*Vector3D*，用于同时描述R、G和B通道的反射率。

- `Vector3D DiffuseBSDF::sample_f` 函数需要将采样光线存储在参数 `wi` 。中。 `wi` 作为指针输入，可以直接修改其值。
- 这部分只需要几行代码。。。

3.2 零反射照明

顾名思义，零反射照明是指光线不经过场景中的任何物体反射就到达相机。换句话说，照明来自光源本身的光。

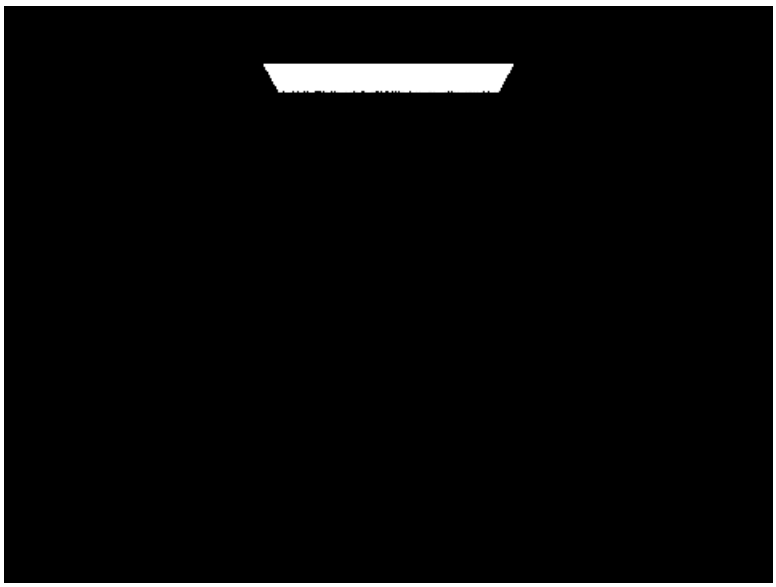
请实现 `zero_bounce_radiance` 函数，该函数输入为一条射线 `Ray` 和 `Intersection` （射线与场景相交的物体）。 函数返回零反射辐照度（相交物体，一般指光源，向外辐射的光的强度）。 完成后，更新 `est_radiance_global_illumination` 函数，返回零辐射辐照度（取代之前使用的的法向着色方法）。

需要用到的代码：

- BSDF 具有 `get_emission()` 函数，它返回物体发射光谱的强度。

编译好代码后，运行以下命令，你将看见下图效果：

```
./pathtracer -t 8 -s 16 -l 8 -m 6 -H -f CBbunny_16_8.png -r 480 360 ../dae/sky/CBbunny.dae
```

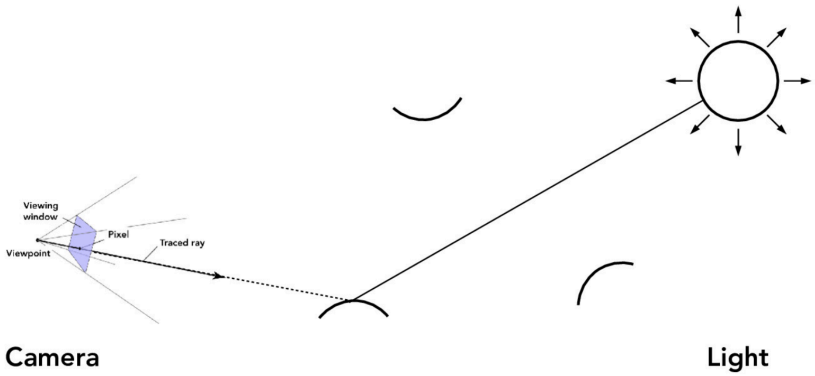


3.3 采用均匀半球采样的直接照明

刚才我们已经定义了一个可用的材质（漫反射BSDF）以及零反射（面积光源）照明，接下来可以实现直接照明了。根据Monte Carlo Ray Tracing，对光线积分可通过随机采样进行估计，这里影响估计精度的关键因素是采样的数量和采样分布。针对光线的采样分布，接下来介绍两种概率密度函数及对应的估计算法：第一种是 `estimate_direct_lighting_hemisphere` ，在半球面上进行均匀采样，本节会重点介绍此方法；其次是 `estimate_direct_lighting_importance` ，针对关键区域进行重要性采样，将在3.4节中实现。

算法流程如下：首先从相机投射一条光线，穿过某个像素，进入场景，与场景中的某个东西相交，根据交点计算像素的颜色。该颜色由交点反射回相机的光量确定。要知道反射光的数量，首先需要知道从各个方向有多少光到达该交点。

1-Bounce Path Connecting Ray to Light



为了估计有多少光到达了该交点，根据渲染方程，需要对到达该点的半球方向进来的所有光进行积分。使用蒙特卡洛估计来进行数值积分：

$$\frac{1}{N}\sum_{j=1}^{N}\frac{f_r(\mathrm{p},\omega_j\rightarrow\omega_r)\,L_i(\mathrm{p},\omega_j)\,\cos\theta_j}{p(\omega_j)}$$

在给定的代码中，p为变量`hit_p`，w_r为`w_out`。对于此任务，我们将在半球内均匀采样光线方向。当前只需关心直接照明，因此只需要检查从`hit_p`出发的**采样光线是否与光源相交**。一旦估计了入射光的数量，就可以使用反射方程（BRDF）来计算出射光的比重了。

实现细节说明

你将需要**更新**`one_bounce_radiance`函数以调用直接照明函数，并**更新**`est_radiance_global_illumination`，把`one_bounce_radiance`添加进去，到此该函数应包含光源经过0次和1次反射的照明效果。

初始给定代码简介：

- `o2w`和`w2o`分别指Object to World和World to Object，是模型坐标系和世界坐标系之间的变换矩阵，关于坐标系间变换参见lecture 4 trasforms。
  - 以上变换基于对象表面交点的局部坐标系创建，后者通过输入的`isect`的法线构建。在局部坐标系下，法线变为$(0,0,1)$，既z为“向上”方向。
  - 这两个变换矩阵互逆，也是互为转置（因为它们是正交的！）
- `hit_p`，射线与表面的交点（需注意交点坐标是在哪个坐标系下）。
- `w_out`，在对象的局部坐标系下的出射方向。关于入射和出射方向，采样时一般都是在物体的局部坐标系下，沿着半球平面的方向，但是在追踪光线时，会涉及到多个交点，需要表示成全局坐标系，具体计算时需要来回做坐标系变换。
- `num_samples`，半球方向采样的数量。
- `EPS_F`，epsilon 常数，用于避免数值精度问题。
- `UniformHemisphereSampler3D::get_sample()`
  - PathTracer 类包含一个`hemisphereSampler`
  - 其返回在半球上均匀随机采样的向量（向量在对象的局部坐标系下）
- `BVHAccel::intersect(Ray& r, Intersection* i)`
- `BSDF::get_emission()`
- `BSDF::f(Vector3D& wo, Vector3D& wi)`
- `Pathtracer::one_bounce_radiance(Ray& r, Intersection& i)`

知识点：

- 两个单位向量之间角度的余弦等于它们的点积。
- 要小心哪些向量在世界坐标系或在对象坐标系下。例如，BSDF 是对象的材质属性 - 那么应该使用哪个坐标空间？当计算射线相交时，应该使用哪个坐标空间？
- 当从现有交点投射新射线时，应该将射线的 `min_t` 设置为一个小的常数 `EPS_F` 。
    - 通过设置 `min_t` 参数，可以避免将射线原点（原表面交点）视为有效交点。
- 需要根据样本数量和每次采样的概率进行归一化。

更新 `est_radiance_global_illumination()` 函数，在其中添加并返回 `estimate_direct_lighting_hemisphere()` 。此时能够使用朗伯漫反射 BSDF 渲染任何 CB 场景（Cornell Box），例如 CBspheres 和 CBbunny。

快速渲染测试

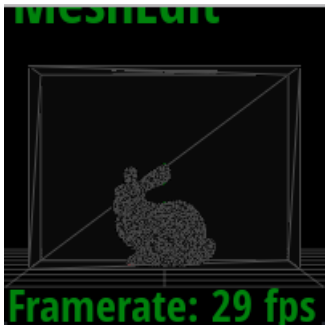
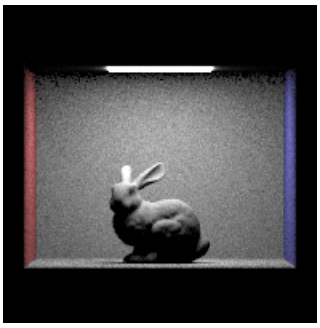
实现到这个阶段，完成高质量甚至中等质量的渲染可能需要很长时间。在开发过程中，你将需要迭代代码/修复错误，需要让程序的快速返回渲染结果。下面再次重述几种加快渲染的方法。

减少采样数、反射次数等。有三个主要参数对渲染时间有很大影响：每像素的采样数（ `-s` 参数）、每个光源的采样数（ `-l` 参数）和每条光路的反射次数（ `-m` 参数，光路的深度）。测试代码时，你可使用最小的采样数。例如，在测试可以随机采样略超过 1 个样本即可。当撰写实验报告时，需要生成最终结果，此时使用较高的样本数，得到高质量的渲染图。

用小视图渲染图像

```
./pathtracer -t 8 -s 64 -l 32 -H -f CBbunny_H_64_32.png -r 100 100 ../dae/sky/CBbunny.dae
```

可以将窗口的分辨率设置为 100 像素 x 100 像素，此时窗口非常小。此时可以使用鼠标进行滚动，放大或缩小场景。滚动时GUI会提供了像左侧图像一样的预览，确定物体大小后，停止滚动，右侧图像为渲染结果。

缩放场景	渲染结果
	

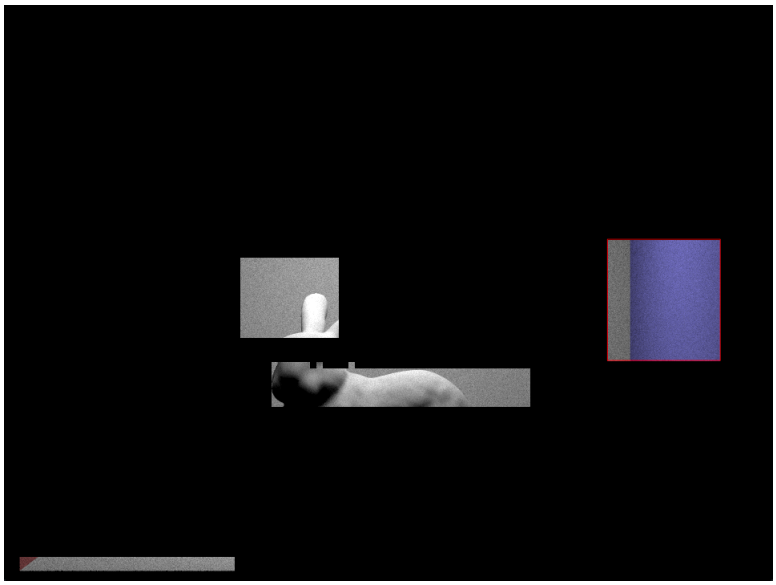
使用单元格渲染功能

按**R 键**开始渲染后，你可以按**C 键**进入单元格渲染模式。此时可以单击鼠标并拖动矩形框，程序将仅针对矩形框运行渲染。这有助于提前抽查场景中的某些区域，而不必等待整个场景渲染完成。

下面是利用单元格渲染场景的不同区域的示例，它能够获取关于渲染的大量信息，且所有单元格组合在一起仅需**秒级时间**。

运行命令如下，请记住你需要使用**R键**启动渲染，然后使用**C键**进入单元渲染模式。

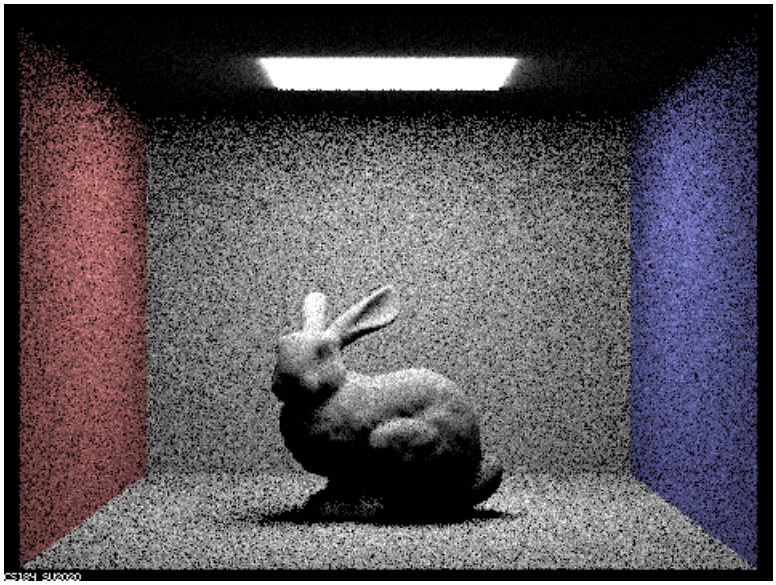
```
./pathtracer -t 8 -s 64 -l 32 -H -f CBbunny_H_64_32.png -r 480 360 ../dae/sky/CBbunny.dae
```



全尺寸图像渲染结果

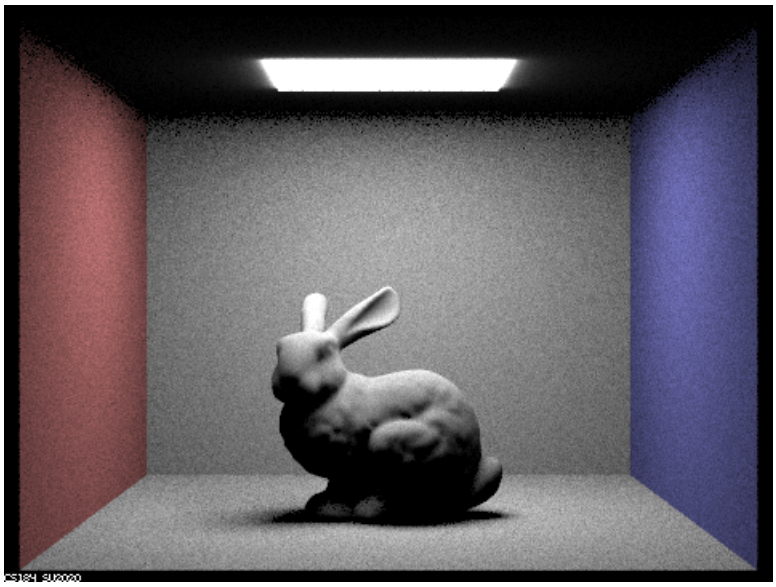
运行如下命令可得到下图效果：

```
./pathtracer -t 8 -s 16 -l 8 -H -f CBbunny_H_16_8.png -r 480 360 ../dae/sky/CBbunny.dae
```



采用更高的采样率可以降低噪声。运行如下命令可得到下图效果：

```
./pathtracer -t 8 -s 64 -l 32 -m 6 -H -f CBbunny_H_64_32.png -r 480 360 ../dae/sky/CBbunny.dae
```



到此你已经能够渲染直接照明的渲染效果了，效果包括面积光源下的阴影（如地板上的影子）和环境光遮蔽（物体表面自遮挡产生的明暗），这些效果是之前Blinn Phong反射模型无法直接获得的。
当前渲染器的一个局限是你只能渲染具有 Lambertian（漫反射） BSDF 的材质，因为尚未实现任何其他 BSDF，本次作业也不要求实现更复杂的BSDF了。对于玻璃、树叶、玉佩、皮肤等质感，需要更丰富的BSDF模型来描述光的散射现象，具体方案有微表面模型等，对此感兴趣的同学可单独与我交流，获取额外作业，进一步提升图形学渲染实践水平。

3.4 重要性采样

从均匀半球采样中得到的结果很噪，虽然它们能够收敛到正确的结果，但是随机性太大，大量采样的光线的贡献度太低。接下来介绍重要性采样方法，提高贡献度高的光线的采样概率。

请实现 `estimate_direct_lighting_importance` 函数，此函数与 `estimate_direct_lighting_hemisphere` 类似，但将直接对光源进行采样，而不是在半球方向上进行均匀采样。

具体而言，对于场景中的每个光源，我们希望对光源和表面点 **`hit_p`** 之间的方向进行采样，也就是说每条采样的射线都连接了光源和表面点，，如此表面点和光源之间没有其他物体，那么这个光源直接将光投射到表面点上。然后只需使用3.3节 中的反射方程来计算表面点出射光的总量。更新 `one_bounce_radiance` 函数，把直接照明从原来的半球均匀采样替换成对面积光源进行重要性采样。

实现细节说明

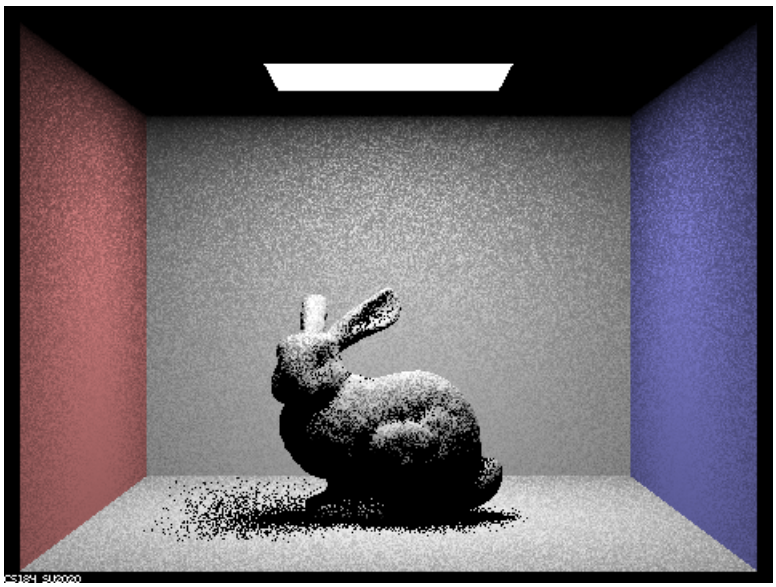
- `scene->lights` 向量存储场景中所有光源的指针
- `SceneLight::is_delta_light()` 返回 `true` 如果光源是点光源
  - 对点光源仅需进行一次采样，因为来自点光源的所有方向样本都是相同的
- `SceneLight::sample_L(Vector3D& p, Vector3D* wi, double* distToLight, double* pdf)` 对光源进行采样，其中
  - 函数返回光线的辐照度 `Vector3D`
  - 函数的输入参数包含下述多个指针，其中 `wi` 是 `p` 和光源之间的采样方向
  - `distToLight` 是一个双精度数，是 `p` 在 `wi` 方向上和光源的距离
  - `pdf` 也是一个双精度数，是 `wi` 的概率密度函数
- `BVHAccel::intersect(Ray& r, Intersection* i)`

- `BSDF::f(Vector3D& wo, Vector3D& wi)`
- `EPS_F` 是 epsilon 常数。

代码测试

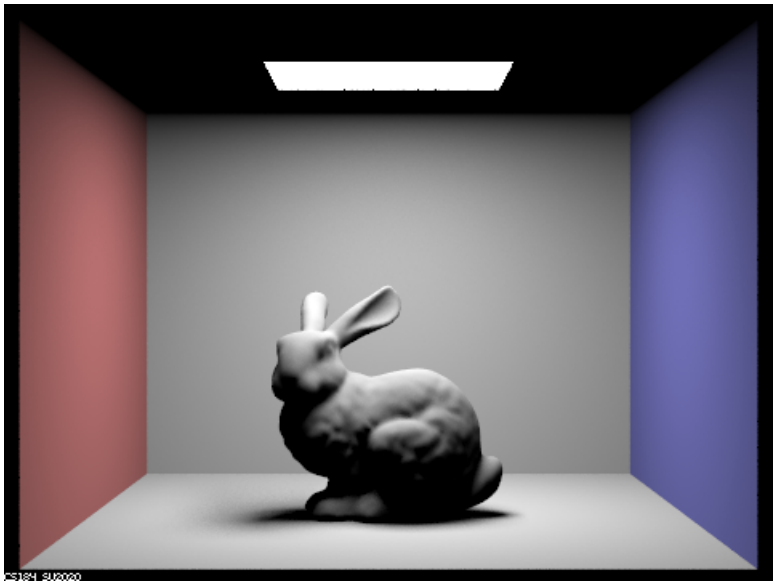
首先使用低采样率检查实现效果：

```
./pathtracer -t 8 -s 1 -l 1 -m 1 -f bunny_1_1.png -r 480 360 ../dae/sky/CBbunny.dae
```



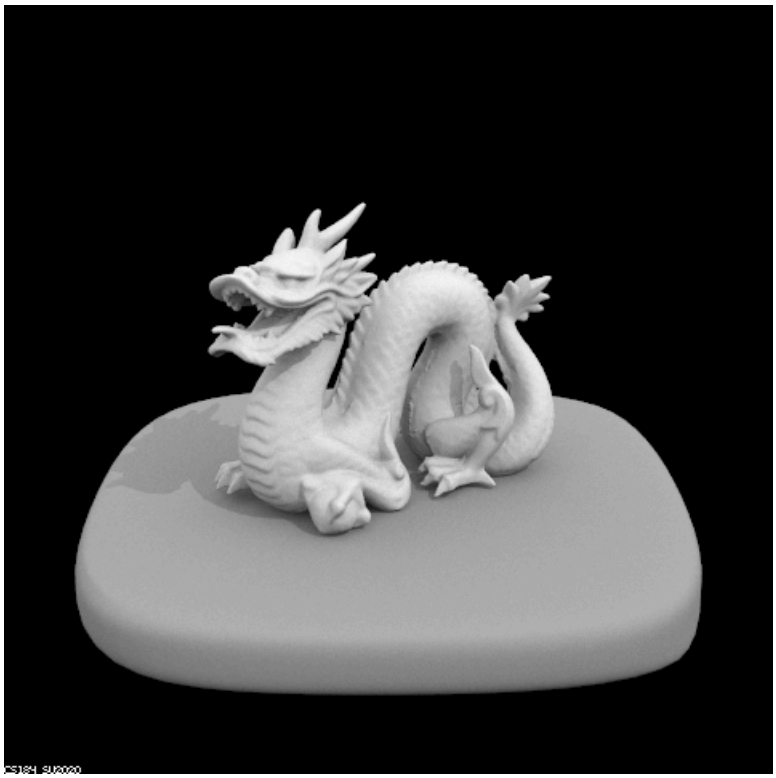
提升采样率后的效果：

```
./pathtracer -t 8 -s 64 -l 32 -m 6 -f bunny_64_32.png -r 480 360 ../dae/sky/CBbunny.dae
```



Dragon场景的渲染结果：

```
./pathtracer -t 8 -s 64 -l 32 -m 6  -f dragon_64_32.png -r 480 480 ../dae/sky/dragon.dae
```



第 4 部分：全局光照

直接光照的结果整体较为暗淡，因为其仅考虑光源的直接照明，并未考虑间接照明。间接照明指的是物体表面被光源直接照明后，有一部分比重的光线会散射到环境中，继续照亮场景中其他地方，这个散射可以一直持续下去。与直接照明的实现过程类似，全局照明也主要考虑两方面：物体表面材质，也就是反射方程；光线的采样。

4.1 使用漫反射 BSDF 进行采样

在此任务中，你将实现函数`DiffuseBSDF::sample_f`来表示一种漫反射材质，该材质在半球上向所有方向均匀反射光。

在第3部分中，你实现了`DiffuseBSDF::f`，其输入入射光线的立体角`wi`和出射光线的立体角`wo`，返回反射率`f(wi -> wo)`。函数`DiffuseBSDF::sample_f`与之有所不同，**它同时需要采样入射光线**，它的输入为参数`wo`和指针`wi`和`pdf`。它执行两个任务：

1. `DiffuseBSDF::sample_f`随机采样入射光线方向，将采样方向结果赋值给`wi`，采样概率赋予`pdf`。
2. `DiffuseBSDF::sample_f`返回`f(wi -> wo)`。

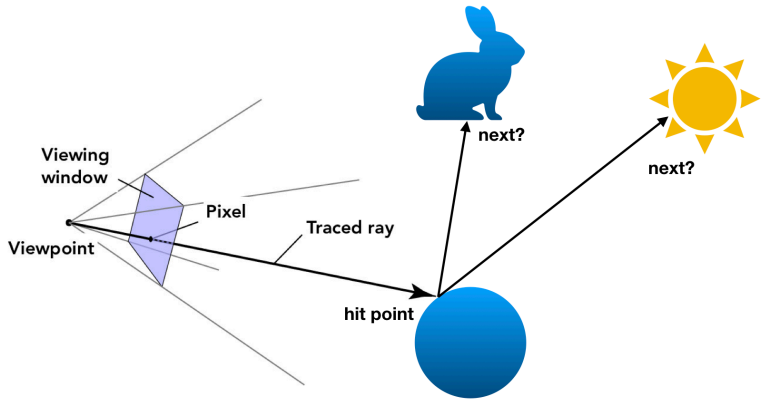
那么如何对入射光线方向进行采样？请查阅`bsdf.h`文件，你会看到`DiffuseBSDF`包含一个`sampler`对象。该`Sampler`从余弦加权的半球分布（cosine-weighted hemisphere distribution）中采样 3D 样本，该概率分布函数在课件中的重要性采样部分有介绍。

4.2 最多N次反射光的全局照明

对于全局光照任务，你的工作是实现函数`sample_f`和`at_least_one_bounce_radiance`函数，并完成*src/pathtracer/pathtracer.cpp*中的`est_radiance_global_illumination`函数。

- `est_radiance_global_illumination` 函数估计一个表面点朝特定方向（此方向如从相机中心朝像素平面投射出来的射线）的总辐照度。其将把零反射照明、一次反射照明（直接照明）和N次反射照明全部累加。前两者之前已介绍，接下来介绍N次反射光的全局照明。
- `at_least_one_bounce_radiance` 函数是第 4 部分的主要工作。此函数会调用 `one_bounce_radiance` 函数，判断当前表面点接收到直接光照的数量，然后再递归调用自身，来计算其他方向的表面点反射到该点的光量。进行递归调用需要新的顶点和方向，因此需要根据当前点的 BSDF 对某个方向进行一次随机采样，同时根据当前点和采样方向连成一条射线，在场景中求得新的交点。
- 如果变量 `isAccumBounces` 设置为true（默认），那么全局照明将会累积 `max_ray_depth` 次反射光。如果设置为false，则将仅返回第 `max_ray_depth` 次反弹的光线。例如，如果 `isAccumBounces=false` 并且， `max_ray_depth =2` ，函数将仅返回第二次反射的光线。此变量用来展示不同光线在经过多次反弹，每次反弹后余下的光量，能够直观的反应光线传播的衰减程度及光线累积的效果，后面有具体的渲染图像。

lecture 13 的附加slide有关于此算法的伪代码。



实现细节说明

与第 3 部分相同，现有代码框架为你提供主要变量和函数：

- `o2w` 和 `w2o` 分别指Object to World和World to Object，是模型坐标系和世界坐标系之间的变换矩阵，关于坐标系间变换参见lecture 4 trasforms。
  - 以上变换基于对象表面交点的局部坐标系创建，后者通过输入的 `isect` 的法线构建。在局部坐标系下，法线变为$(0,0,1)$，既z为“向上”方向。
  - 这两个变换矩阵互逆，也是互为转置（因为它们是正交的！）
- `hit_p` ，射线与表面的交点（需注意交点坐标是在哪个坐标系下）。
- `w_out` ，在对象的局部坐标系下的出射方向。
- `BSDF::sample_f(Vector3D& w_out, Vector3D* w_in, double* pdf)` 在4.1节中介绍了
- `BVHAccel::intersect(Ray&, Intersection*)`
- `at_least_one_bounce_radiance(Ray&, Intersection&)`
- `max_ray_depth` 在 `raytrace_pixel` 中修改。
- 在计算交点时，给计算值加上偏置值 `EPS_F` ，避免一些数值问题。
- 对方向等矢量进行归一化等
- 不要忘了在Monte Carlo估计式里除以概率项。
- 最容易出错的是坐标变换，需要清楚当前坐标是在世界坐标系还是局部坐标系。

全局照明效果

isAccumBounces	m=0	m=1	m=2	m=3	m=4	m=5
False						
True						

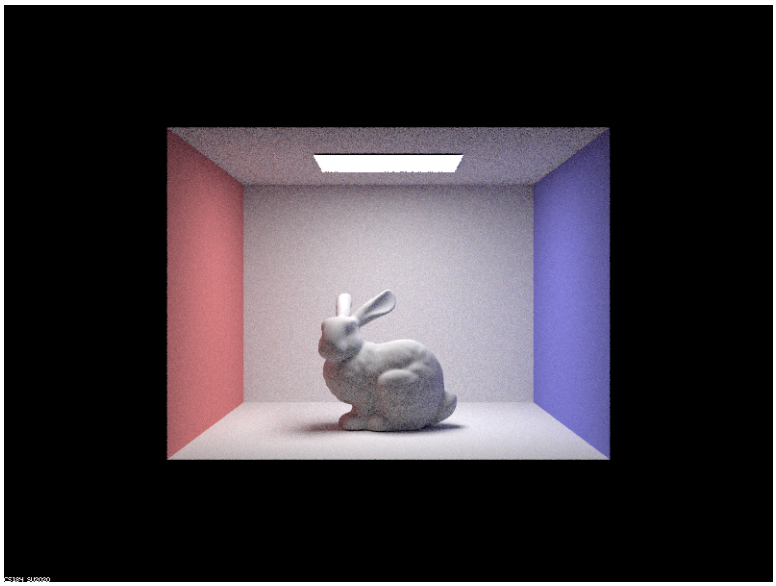
请放大观看。m=0是零反射照明，光源直接进入虚拟相机传感器，图像中只有光源是亮的，其他地方黑的；m=1是直接光照的效果，可以看到房顶除光源是暗的，因为光源无法直接照到；m>=2时，随着m增加，场景越来越亮，因为累积了更多的反射光，但是当m=5时，反射光非常微弱，累积结果变化不大。

4.3 Russian Roulette 全局照明

在4.2节中，我们实现了全局照明，其终止于`max_ray_depth`次光线反弹。即使使用非常大的`max_ray_depth`，估计结果仍然是有偏差，虽然多次反弹后光线的贡献度极低。为了保证Monte Carlo估计是无偏的，我们介绍了Russian Roulette做为光线追踪的终止条件，该方法的无偏估计性质见课件。Russian Roulette 在`at_least_one_bounce_radiance`中实现。

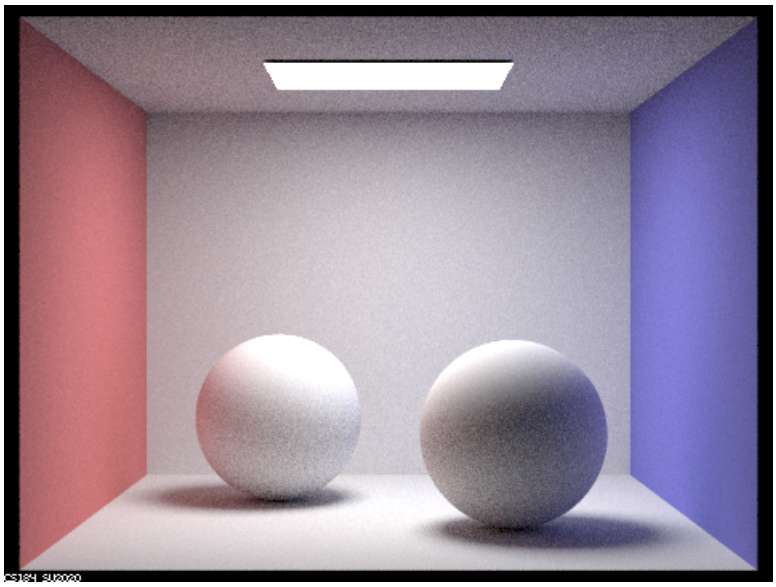
- Russian Roulette提供了一种无偏的随机终止方法。
  - 理论上，终止概率可以是任意的——但我们建议使用 0.3 或 0.4 之间的概率值。
  - 需要注意的是，根据你选择的值，你的渲染图像可能看起来与本说明或你的同学的结果不同，但是从统计意义来讲，期望是相同的。
- `max_ray_depth`如果`> 1`，则间接照明生效，此时至少需要进行一次光线反弹（无论终止概率如何）。
- 如果新光线不与场景相交，那么无需递归下去。
- `coin_flip(double p)`函数定义在`random_util.h`，此函数以概率`p`返回`true`

设置`m=32`，根据新添加的Russian Roulette终止条件，运行结果如下：



虽然目前仍然只使用漫反射 BSDF进行渲染，但是已经能够在场景中看到一些漂亮的颜色渗色。如下图所示球面上有墙壁的倒影。

```
./pathtracer -t 8 -s 64 -l 16 -m 5 -r 480 360 -f spheres.png ../dae/sky/CBspheres_lambertian.dae
```



注意：不使用漫反射 BSDF 的物体外观会完全是黑色的，如下图所示。



以下是一些参考场景，可以用于渲染作业。

`/dae/sky/CBbunny.dae`

`/dae/sky/CBempty.dae`

`/dae/sky/CBspheres_lambertian.dae`

`/dae/sky/bench.dae`

`/dae/sky/blob.dae`

`/dae/sky/bunny.dae`

`/dae/sky/dragon.dae`

`/dae/sky/wall-e.dae`